PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

IMAGE BASED BREED RECOGNITION FOR CATTLE AND BUFFALOES OF INDIA

A PROJECT REPORT

Submitted by

AMISHA – 20221COM0170

KAVYA D K - 20221COM0179

Under the guidance of,

Dr. MURALI PARAMESHWARAN

BACHELOR OF TECHNOLOGY

IN

COMPUTER ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SPECIALIZATION DIVISION)

BONAFIDE CERTIFICATE

Certified that this report "**IMAGE BASED BREED RECOGNITION FOR CATTLE AND BUFFALOES OF INDIA"** is a bonafide work completed by **AMISHA (20221COM0170), KAVYA D K (20221COM0179)**, who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER ENGINEERING** during 2025-26.

Dr. Murali Parmeshwaran	**Ms. Sumitha Guddin**	**Dr. Sampath A K** **Dr. Geetha**	**Dr. Gopal Krishna Shyam**
Project Guide PSCE Presidency University	Program Project Coordinator PSCE Presidency University	School Project Coordinators PSCE Presidency University	Head of the Department PSCE Presidency University

Dr. Duraipandian N	**Dr. Shakkeera L**
Dean PSCE Presidency University	Associate dean PSCE Presidency University

Examiners

Sl. No	Name of the Examiner	Signature	Date

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF ENGINEERING

DECLARATION

We the students of final year B.Tech in **COMPUTER ENGINEERING** at Presidency University, Bengaluru, named **AMISHA (20221COM0170), KAVYA D K (20221COM0179),** hereby declare that the project work titled **"IMAGE BASED BREED RECOGNITION FOR CATTLE AND BUFFALOES OF INDIA"** has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in **COMPUTER ENGINEERING** during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

AMISHA **USN: 20221COM0170**

KAVYA D K **USN: 20221COM0179**

PLACE: BENGALURU

DATE:

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr. Murali Parameshwaran, Associate Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Gopal Krishna Shyam, Head of the Department**, Presidency School of Computer Science and Engineering Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N,** Dean and **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering (Specialization Division) and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **MS. Sumitha Guddin** Program Project Coordinator, and **Dr. Sampath A K,** and **Dr. Geetha A,** School Program Project Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

AMISHA

KAVYA D K

Abstract

The project focuses on developing an automated system for the recognition of Indian cattle and buffalo breeds using image-based deep learning techniques. Traditional breed identification methods are manual, time-consuming, and require expert knowledge, often leading to inaccuracies. To address this challenge, the proposed system utilizes Convolutional Neural Networks (CNNs) to classify livestock breeds from images efficiently. By leveraging modern deep learning approaches, the system aims to support farmers, researchers, and conservationists in accurate and quick breed identification.

The system is built using multiple pre-trained CNN architectures, including MobileNetV2, MobileNetV3, DenseNet121, EfficientNetV2S, EfficientNetB3, and ConvNeXt, with transfer learning techniques applied to improve performance. A diverse dataset of cattle and buffalo images was used for training and validation. Image preprocessing and data augmentation techniques were employed to enhance model generalization and robustness. The models were evaluated using performance metrics such as accuracy, precision, recall, and F1- score to ensure reliable and effective classification results.

Furthermore, the trained models were optimized using TensorFlow Lite with post-training quantization to enable efficient deployment on web and mobile platforms. A user-friendly web application was developed using Flask, allowing users to upload images and receive real-time breed predictions along with confidence scores. This project contributes to livestock management and conservation efforts by providing an accessible, scalable, and efficient breed recognition solution, with potential for future improvements such as dataset expansion and mobile application development.

Table of Content

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	iv
	Abstract	1
	List of Figures	4
	List of Tables	5
	Abbreviations	6
1.	Introduction 1.1 Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of project report	7-11
2.	Literature review	12-14
3.	Methodology	15-20
	Project management 4.1 Project timeline 4.2 Risk analysis 4.3 Project budget	21-28
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing devices 5.5 Designing units 5.6 Standards 5.7 Mapping with IoTWF reference model layers 5.8 Domain model specification	29-42

	5.9 Communication model 5.10 IoT deployment level 5.11 Functional view 5.12 Mapping IoT deployment level with functional view 5.13 Operational view 5.14 Other Design	
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software development tools 6.3 Software code 6.4 Simulation	43-50
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	51-58
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	59-60
9.	Conclusion	61
	References	62
	Appendix	

List of Figures

3.1	Target to Source	16
3.2	The arrangement table	16
3.3	SDLC phases	17
4.7	Timeline Gantt chart	23
5.2	Block diagram	31
5.3	System flow chart	31

List of Table

1.6	SDGS Table	11
2.1	Summary of Literature review	14
4.1	Project Timeline	23
5.7	Mapping with IOTWF reference model	35
5.12	Mapping with IOT deployment	39
7.3	Test Results	57

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
IoT	Internet of Things
VGG	Visual Geometry Group (Model)
ResNet	Residual Network
API	Application Programming Interface
UI	User Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RGB	Red, Green, Blue
IDE	Integrated Development Environment
URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
SQL	Structured Query Language
ReLU	Rectified Linear Unit
F1-Score	Harmonic Mean of Precision and Recall
SDLC	Software Development Life Cycle
IoTWF	Internet of Things World Forum

Chapter 1
Introduction

With the advancement of artificial intelligence and computer vision, image-based breed recognition has emerged as a modern and efficient alternative. This approach utilizes digital images of animals and processes them using deep learning models to automatically classify breeds based on distinct physical features such as body shape, coat color, horn structure, and facial patterns. By leveraging convolutional neural networks and other advanced algorithms, the system can learn subtle differences between breeds that may not be easily distinguishable by the human eye.

An automated image-based system for cattle and buffalo breed recognition offers several advantages, including faster decision-making, reduced dependency on expert knowledge, and improved accuracy. It can be integrated into mobile or web applications, enabling farmers, veterinarians, and researchers to identify breeds in real-time using simple image inputs. Such systems can also contribute to better breeding programs, disease management, and conservation of indigenous species.

1.1 Background

The rapid growth of digital technologies has opened new possibilities in this area. With the increasing availability of smartphones and internet connectivity, capturing and sharing animal images has become easier than ever. At the same time, advancements in machine learning, particularly in the field of computer vision, have enabled systems to analyze visual data with high precision. Image-based recognition systems are now widely used in various domains, including facial recognition, medical diagnosis, and plant disease detection, demonstrating their effectiveness in pattern identification tasks.

1.2 Statistics of project

India is home to a wide variety of recognized cattle and buffalo breeds, each adapted to specific climatic conditions and production needs. However, a large portion of the livestock population consists of non-descript or crossbred animals, making accurate breed identification more challenging. Studies indicate that improper breed identification can lead to reduced productivity, inefficient breeding practices, and loss of valuable indigenous genetic traits.

In recent years, the adoption of artificial intelligence in agriculture has shown promising growth. Research in computer vision has demonstrated that deep learning models, particularly convolutional neural networks, can achieve high accuracy levels—often exceeding 90%—in image classification tasks when trained on well-curated datasets. Similar approaches applied to livestock have reported strong performance .

1.3 Prior existing technologies

The emergence of deep learning marked a significant shift in this domain. Convolutional neural networks (CNNs) enabled automatic feature extraction directly from images, eliminating the need for manual intervention. Pre-trained models such as VGG, ResNet, and MobileNet began to be adapted for livestock-related tasks, offering improved performance and robustness. These models demonstrated the ability to capture complex visual patterns and achieve higher accuracy compared to earlier techniques

1.4 Proposed approach

Following preprocessing, a deep learning-based model is designed using convolutional neural networks (CNNs). Instead of building a model from scratch, transfer learning is utilized by adopting pre-trained architectures such as VGG, ResNet, or MobileNet. These models are fine-tuned on the livestock dataset to learn breed-specific features such as body patterns, facial structure, horn shape, and coat texture. Multiple models may be trained and combined using ensemble techniques to improve overall prediction accuracy and reliability.

The trained model is then integrated into a web-based or mobile-based application. The user interface allows users to upload an image of a cattle or buffalo, which is then processed by the backend system. The model analyzes the image and outputs the predicted breed along with a confidence score. Additional features such as breed information, characteristics, and recommendations can also be displayed to enhance usability.

1.5 Objectives

One of the key objectives is to collect and prepare a high-quality dataset containing images of various cattle and buffalo breeds. This includes ensuring diversity in terms of environmental conditions, image angles, and animal features so that the model can perform effectively in real-world scenarios.

Another important objective is to implement and train deep learning models capable of extracting meaningful visual features from images. The system should be able to distinguish between breeds based on characteristics such as body structure, coat patterns, and facial features, while maintaining high accuracy and consistency.

The project also aims to explore the use of transfer learning by utilizing pre-trained models and fine-tuning them for breed classification tasks. This helps in reducing training time and improving model performance, especially when working with limited datasets.An additional objective is to develop a user-friendly interface, such as a web or mobile application, that allows users to upload images and receive instant breed predictions. The system should be simple to use so that farmers and non-experts can easily benefit from it.

Research Motivation

Another key factor driving this research is the importance of accurate breed identification in improving productivity and sustainability. Different breeds exhibit varying characteristics in terms of milk yield, disease resistance, and adaptability to environmental conditions. Without proper identification, farmers may face challenges in selecting suitable animals for breeding, which can negatively impact overall farm performance and economic returns.

The rapid advancement of artificial intelligence, especially in computer vision, provides a strong opportunity to address these challenges. Deep learning models have demonstrated exceptional capability in recognizing complex visual patterns, making them highly suitable for distinguishing subtle differences between animal breeds. Leveraging these technologies can significantly enhance the accuracy and efficiency of breed classification systems.

1.6 SDGs

Table 1.6

SDG Goal	SDG Title	Relevance to the Project
SDG 2	Zero Hunger	Supports improved livestock productivity by enabling accurate breed identification, which helps farmers increase milk production and food availability.
SDG 8	Decent Work and Economic Growth	Enhances farmers' income by improving decision-making in breeding and livestock management, leading to better economic outcomes.
SDG 9	Industry, Innovation and Infrastructure	Promotes the use of advanced technologies such as artificial intelligence and computer vision in agriculture, encouraging innovation.
SDG 12	Responsible Consumption and Production	Helps in efficient resource utilization by identifying suitable breeds for specific environments, reducing wastage and improving sustainability.
SDG 13	Climate Action	Encourages the selection of climate-resilient breeds, which can adapt better to environmental changes and reduce agricultural risks.
SDG 15	Life on Land	Contributes to the conservation of indigenous cattle and buffalo breeds by enabling accurate identification and documentation.

1.7 Overview of project report

The report begins with an introduction that explains the importance of livestock in agriculture and the need for accurate breed identification. It highlights the limitations of traditional methods and introduces the concept of using artificial intelligence to automate the process. The background section further elaborates on existing practices and technological developments that have influenced this research.

Following this, the report reviews prior existing technologies and identifies the gaps that motivate the need for a more efficient and accessible system. It then outlines the objectives and research motivation, providing a clear direction for the study. These sections establish the foundation for the proposed approach.

The methodology section describes the step-by-step process involved in building the system, including data collection, preprocessing, model selection, training, and evaluation. It explains how deep learning models are applied to extract features from images and perform breed classification with high accuracy. The integration of the model into a user-friendly application is also discussed.

The results and analysis section presents the performance of the developed system using relevant evaluation metrics. It compares different models, highlights improvements, and discusses the effectiveness of the approach under various conditions. Any challenges faced during implementation are also addressed.

Chapter 2

Literature review

Livestock breed identification has traditionally relied on manual observation techniques, where experts classify cattle and buffalo breeds based on physical characteristics such as body size, coat color, and horn structure. Although widely practiced, this approach is subjective and depends heavily on the experience of the observer, which can lead to inconsistent results [1].

With the advancement of digital technologies, early research shifted toward automated image processing techniques. These methods focused on extracting handcrafted features such as texture, shape, and color from animal images. Machine learning algorithms, including support vector machines and k-nearest neighbors, were then used for classification tasks. However, these approaches required significant manual intervention and were limited in handling variations in environmental conditions [2].

The emergence of deep learning, particularly convolutional neural networks (CNNs), has significantly improved image-based recognition systems. CNNs are capable of automatically learning hierarchical features from raw image data, eliminating the need for manual feature extraction. Studies have shown that deep learning models achieve higher accuracy and robustness compared to traditional methods in image classification tasks [3].

Recent research has explored the application of pre-trained deep learning models such as VGG, ResNet, and MobileNet for livestock-related tasks. These models, when fine-tuned using domain-specific datasets, have demonstrated strong performance in recognizing animal breeds and species. Transfer learning has become a widely adopted approach, as it reduces computational cost and improves efficiency, especially when large labeled datasets are not available [4].

In the specific domain of cattle and buffalo breed recognition, researchers have reported promising results using deep learning techniques. Some studies have achieved high classification accuracy by training models on diverse datasets containing images captured under different conditions. Additionally, ensemble learning methods, which combine multiple models, have been used to enhance prediction performance and system reliability [5].

Despite these advancements, several challenges remain. Limited availability of high-quality annotated datasets, variations in lighting and background, and differences in animal posture can affect model performance. Researchers emphasize the need for more comprehensive datasets and robust models to ensure consistent accuracy in real-world applications [6].

Overall, existing literature highlights a transition from traditional manual methods to intelligent, automated systems based on deep learning. While significant progress has been made, and improvements are required.

2.1 Summary of Literatures reviewed

Table 2.1

S.No	Approach Used	Key Contribution	Limitations
1	Traditional Manual Methods	Identification based on physical features like size, color, and horn shape using expert knowledge	Subjective, time-consuming, and inconsistent results
2	Documentation & Pedigree Records	Maintains breed lineage and historical data for classification	Requires proper record-keeping, not accessible to all farmers
3	Image Processing with Handcrafted Features	Uses features like texture, shape, and color for classification	Limited accuracy, sensitive to environmental variations
4	Machine Learning Algorithms	Applies models like SVM, KNN for automated classification	Requires manual feature extraction and lacks robustness
5	Deep Learning (CNN Models)	Automatically extracts features and improves classification accuracy	Needs large datasets and high computational power
6	Transfer Learning Approaches	Uses pre-trained models to reduce training time and improve performance	Performance depends on dataset quality and fine-tuning
7	Ensemble Learning Techniques	Combines multiple models to enhance accuracy and reliability	Increased complexity and computational cost
8	Real-Time Applications (Web/Mobile)	Enables practical use by farmers through easy image input systems	Performance affected by image quality and internet access

Chapter 3
Methodology

The methodology of this project is designed to develop an efficient and accurate system for identifying cattle and buffalo breeds using image data. It follows a structured approach that includes data collection, preprocessing, model development, training, evaluation, and deployment.

The first step involves collecting a comprehensive dataset of cattle and buffalo images from various sources such as farms, online repositories, and field surveys. The dataset is prepared to include multiple breeds with variations in lighting conditions, angles, backgrounds, and animal postures to ensure that the model can generalize well in real-world scenarios.

Once the dataset is collected, preprocessing techniques are applied to improve the quality and consistency of the images. This includes resizing images to a standard dimension, normalizing pixel values, and removing noise or irrelevant background information where possible. Data augmentation methods such as rotation, flipping, zooming, and brightness adjustment are used to artificially expand the dataset and reduce the risk of overfitting.

The next stage focuses on model development using deep learning techniques. A convolutional neural network (CNN) is selected as the core model due to its effectiveness in image classification tasks. Transfer learning is applied by using pre-trained models such as VGG, ResNet, or MobileNet, which are fine-tuned on the prepared dataset. This approach allows the system to learn complex visual features such as coat patterns, facial structures, and body shapes that differentiate breeds.

After model selection, the dataset is divided into training, validation, and testing sets. The model is trained using the training data while the validation set is used to tune hyperparameters and prevent overfitting. During training, optimization techniques and loss functions are applied to improve model performance. The final evaluation is conducted on the test dataset using metrics such as accuracy, precision, recall, and F1-score to assess the effectiveness of the model.

Once the model achieves satisfactory performance, it is integrated into a user-friendly application. The system allows users to upload images of cattle or buffalo, which are then processed by the trained model to predict the breed. The output includes the predicted class along with a confidence score, making the system practical for real-world use.

Finally, the system is tested under different conditions to ensure reliability and robustness. Continuous improvement can be achieved by updating the dataset and retraining the model periodically. This methodology ensures the development of a scalable, accurate, and accessible solution for livestock breed recognition.

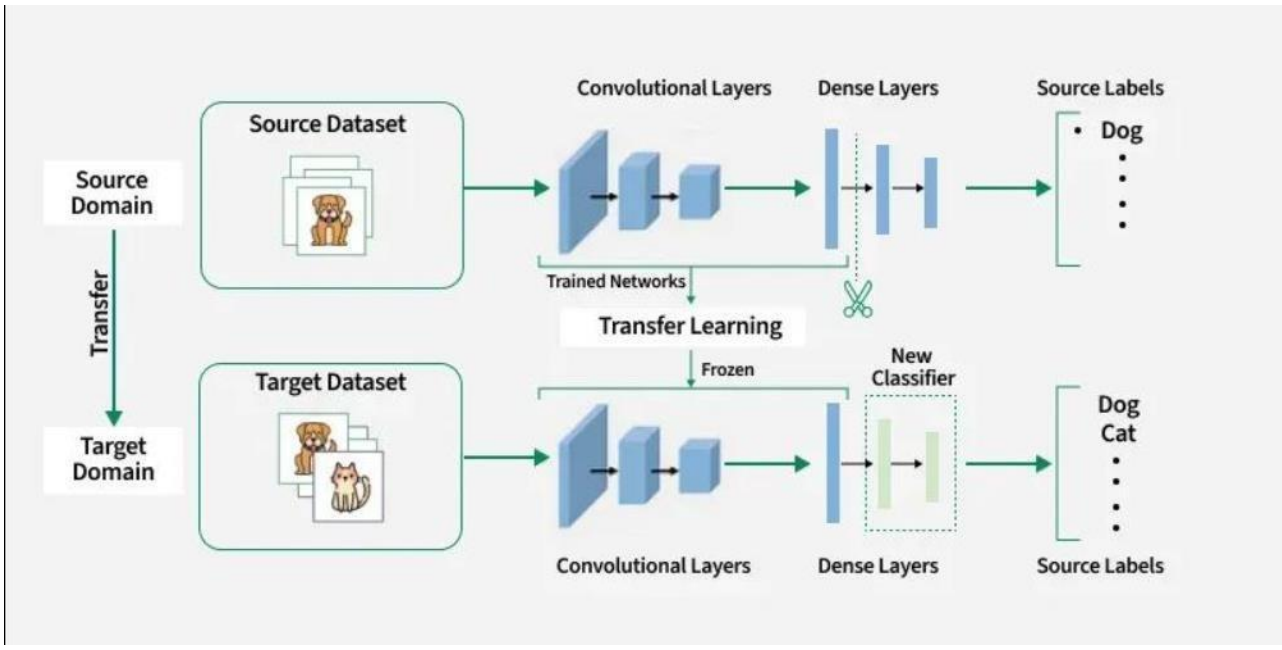


Figure 3.1 Target to Source

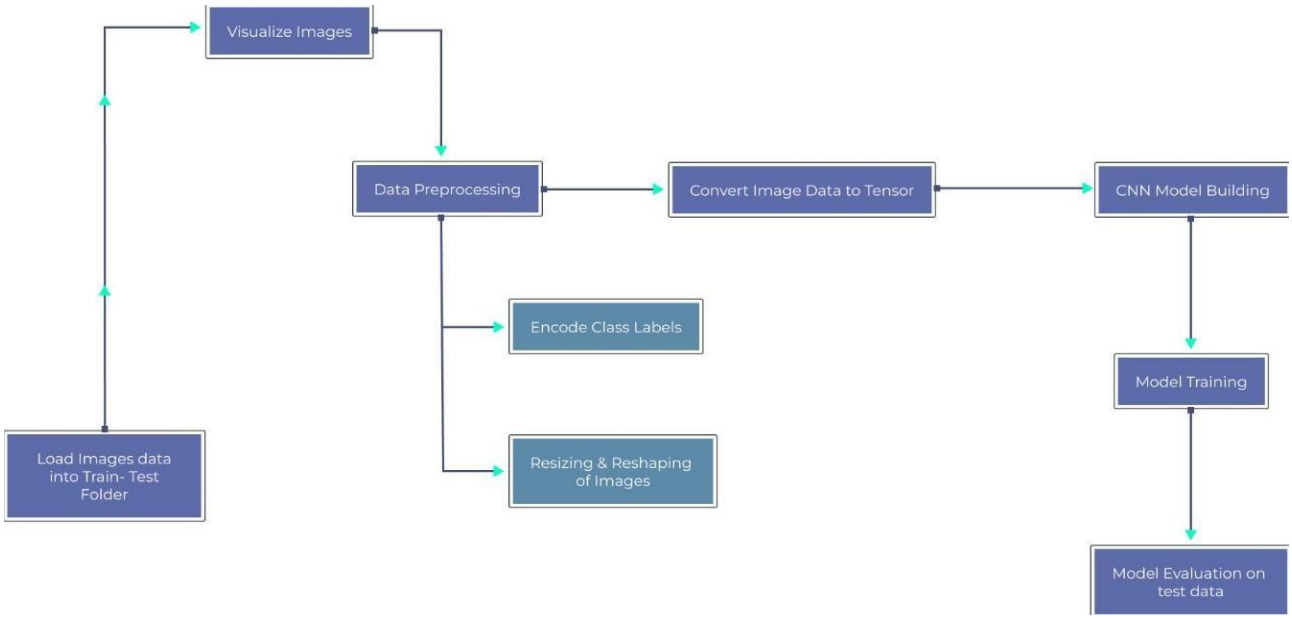


figure :3.2 the arrangement

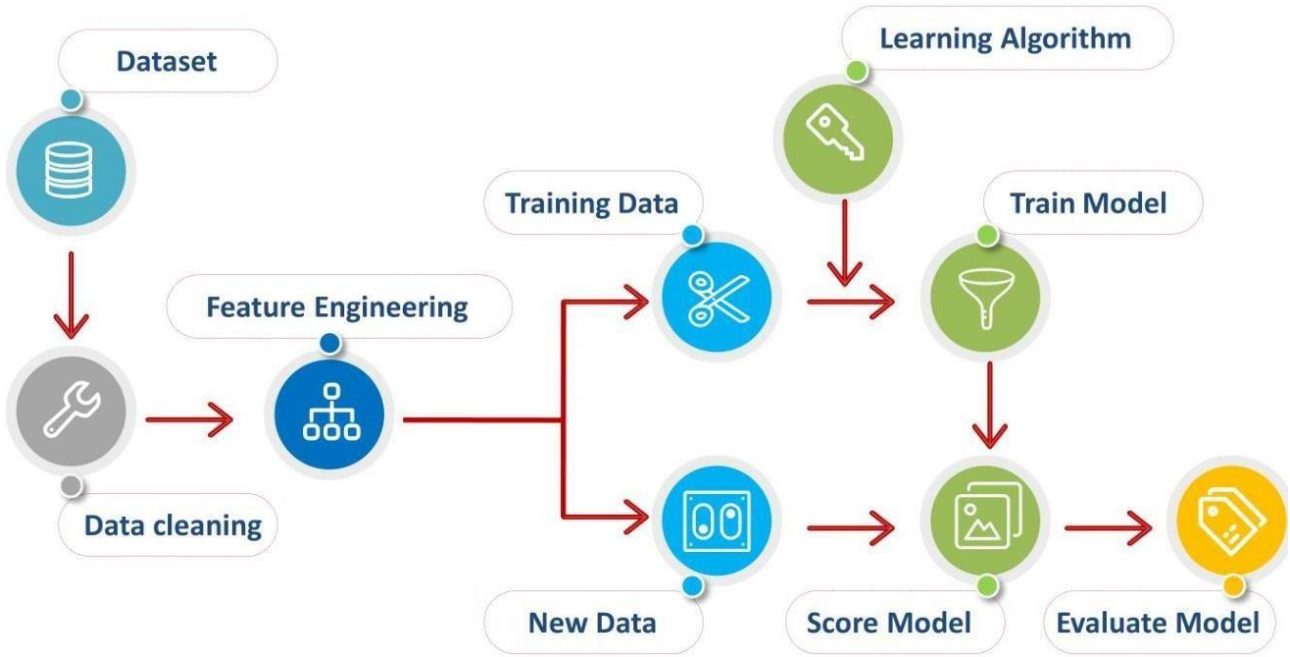


Figure: 3.4 SDLC phases

Models Used: Image-Based Breed Recognition for Cattle and Buffalo

The proposed system utilizes advanced deep learning models to accurately classify cattle and buffalo breeds from images. These models are selected based on their strong performance in image recognition tasks and their ability to extract complex visual features.

1. Convolutional Neural Network (CNN)

A Convolutional Neural Network forms the core of the system. It is designed to automatically learn important features from images such as edges, textures, patterns, and shapes. CNNs consist of multiple layers including convolutional layers, pooling layers, and fully connected layers, which work together to identify distinguishing characteristics of different breeds.

2. VGG Model

The VGG architecture is known for its simplicity and uniform structure. It uses small convolutional filters and deep layers to capture detailed features from images. In this project, a pre-trained VGG model is fine-tuned to adapt to cattle and buffalo breed classification, enabling the system to learn fine-grained visual differences.

3. ResNet (Residual Network)

ResNet introduces residual connections that help in training very deep networks without performance degradation. This model is effective in capturing complex patterns and avoids issues such as vanishing gradients. It improves classification accuracy by allowing deeper feature extraction.

4. MobileNet

MobileNet is a lightweight and efficient deep learning model designed for mobile and embedded applications. It uses depthwise separable convolutions to reduce computational cost while maintaining good accuracy. This makes it suitable for real-time breed recognition systems deployed on smartphones or web applications.

5. Transfer Learning

Instead of training models from scratch, transfer learning is applied by using pre-trained versions of VGG, ResNet, and MobileNet. These models are already trained on large datasets and are fine-tuned using the livestock dataset. This approach reduces training time and improves performance, especially when the dataset is limited.

6. Ensemble Model

To further improve accuracy, multiple models can be combined using ensemble techniques. The predictions from different models are aggregated to produce a final result, increasing reliability and reducing errors.

Software

The development of the image-based breed recognition system requires a combination of programming environments, libraries, frameworks, and tools that support data processing, model development, and deployment. The following software components are used to build and implement the system effectively.

1. Programming Language (Python)

Python is used as the primary programming language due to its simplicity, readability, and extensive support for machine learning and image processing. It provides a wide range of libraries that make development faster and more efficient.

2. Deep Learning Frameworks

Frameworks such as TensorFlow and PyTorch are used to build and train deep learning models. These frameworks provide powerful tools for designing neural networks, performing computations, and optimizing model performance.

3. Image Processing Libraries

Libraries like OpenCV and PIL (Python Imaging Library) are used for handling and preprocessing images. These tools help in tasks such as resizing, filtering, normalization, and data augmentation.

4. Data Handling and Numerical Libraries

NumPy and Pandas are used for efficient data manipulation and numerical computations. They assist in managing datasets, performing mathematical operations, and organizing data for training and testing.

5. Development Environment

Integrated Development Environments (IDEs) such as Jupyter Notebook, Google Colab, or Visual Studio Code are used for writing, testing, and debugging code. These platforms provide an interactive environment for experimentation and model development.

6. Web

For deployment, frameworks like Flask or Django are used to create a user interface where users can upload images and view predictions. These frameworks help in connecting the trained model with the front-end interface.

7. Visualization Tools

Libraries such as Matplotlib and Seaborn are used to visualize data, training progress, and model performance. Graphs and plots help in analyzing accuracy, loss, and other evaluation metrics.

8. Version Control System

Git is used for version control to manage code changes and track project development. It allows collaboration and ensures proper maintenance of different versions of the project.

phases of the System: Image-Based Breed Recognition for Cattle and Buffalo

1. Data Collection Phase

In this phase, images of different cattle and buffalo breeds are gathered from various sources such as farms, online datasets, and field observations. The goal is to collect a diverse set of images that represent different breeds under varying environmental conditions.

2. Data Preprocessing Phase

The collected images are processed to ensure consistency and quality. This includes resizing images to a standard size, normalizing pixel values, and removing unnecessary noise. Techniques such as rotation, flipping, and zooming are applied to increase dataset diversity and improve model performance.

3. Model Development Phase

A deep learning model is designed using convolutional neural networks. Pre-trained models like VGG, ResNet, or MobileNet are selected and fine-tuned to recognize breed-specific features. This phase focuses on building a model that can learn and differentiate between various breeds effectively.

4. Training Phase

The prepared dataset is divided into training and validation sets. The model is trained using labeled images, allowing it to learn patterns and features associated with each breed. During this phase, parameters are adjusted to improve accuracy and reduce errors.

5. Testing and Evaluation Phase

After training, the model is tested using unseen data to evaluate its performance. Metrics such as accuracy, precision, recall, and F1-score are used to measure how well the system performs in classifying breeds correctly.

6. Deployment Phase

Once the model achieves satisfactory results, it is integrated into a user-friendly application. Users can upload images, and the system processes them to provide breed predictions along with confidence scores.

7. Maintenance and Improvement Phase

In this final phase, the system is continuously monitored and updated. New data can be added, and the model can be retrained to improve accuracy.

Chapter 4

Project Management

4.1 Project Timeline

The development of the image-based breed recognition system for cattle and buffalo is carried out in a systematic and well-planned sequence of stages over a defined period. Initially, the project begins with problem identification and background research, where the need for an automated breed recognition system is understood and existing solutions are studied. This phase helps in defining the objectives and scope of the project.

Following this, the data collection phase is undertaken, during which a diverse set of images representing different cattle and buffalo breeds is gathered from multiple sources. Once the data is collected, it is prepared through preprocessing steps such as resizing, normalization, and augmentation to ensure consistency and improve the quality of the dataset.

After preparing the dataset, the focus shifts to model development. In this stage, suitable deep learning models are selected and designed to perform image classification. These models are then trained using the prepared dataset, allowing them to learn distinguishing features of different breeds. The training process is carefully monitored and adjusted to achieve optimal performance.

Once the model is trained, it undergoes testing and evaluation using unseen data to measure its accuracy and reliability. Based on the results, necessary improvements are made to enhance performance. Following successful evaluation, the system is integrated into a user-friendly application, enabling users to upload images and receive predictions.

In the final stages, the system is deployed for practical use, and the entire project is documented in detail. The report includes all aspects of the work, from methodology to results, along with diagrams and explanations. This structured timeline ensures the smooth execution of the project from initial concept to final implementation.

Table 4.1

Phase	Activity	Duration	Description
Phase 1	Problem Identification & Research	Week 1 – Week 2	Understanding the problem, reviewing existing methods, and defining project scope
Phase 2	Data Collection	Week 3 – Week 5	Gathering images of cattle and buffalo breeds from different sources
Phase 3	Data Preprocessing	Week 6 – Week 7	Cleaning, resizing, normalizing images and applying augmentation techniques
Phase 4	Model Development	Week 8 – Week 10	Selecting and designing CNN models (VGG, ResNet, MobileNet)
Phase 5	Model Training	Week 11 – Week 13	Training the model using dataset and optimizing performance
Phase 6	Testing & Evaluation	Week 14 – Week 15	Evaluating model accuracy using test data and performance metrics
Phase 7	System Development	Week 16 – Week 17	Building user interface (web/mobile application)
Phase 8	Deployment	Week 18	Integrating model with application and making system usable
Phase 9	Documentation & Final Report	Week 19 – Week 20	Preparing project report, diagrams, and presentation

Project Timeline Gantt chart

Deployments								week7	
testing							week6		
integration						week6			
Model training					week5				
Software development				week4					
Training			week3						
Requirements assemble		week2							
analyzing	Week 1								

Table 4.1: Gantt chart

1. Identifying Needs

The need for this project arises from the challenges faced in accurately identifying cattle and buffalo breeds using traditional methods. In many agricultural environments, breed identification is still performed manually based on physical observation and personal experience. This approach is not only time-consuming but also prone to errors and inconsistencies, especially when dealing with similar-looking breeds or crossbred animals.

There is a growing demand for a reliable and automated system that can assist farmers, veterinarians, and researchers in identifying breeds quickly and accurately. Proper breed identification plays a crucial role in improving milk production, selecting suitable animals for breeding, and managing livestock effectively. Without accurate identification, farmers may face reduced productivity and inefficient resource utilization.

Another important need is accessibility. Many existing solutions, such as genetic testing, require specialized equipment and are costly, making them impractical for small-scale farmers. Therefore, there is a requirement for a cost-effective and easy-to-use system that can operate using commonly available devices like smartphones.

The rapid advancement of artificial intelligence and the increasing availability of image data create an opportunity to address these challenges. By utilizing image-based recognition systems, it becomes possible to provide instant and accurate results with minimal user effort. This approach can significantly reduce dependency on expert knowledge and make breed identification more accessible to a wider audience.

4.2 Risk Analysis

1. Data-Related Risks

One of the major risks is the availability and quality of the dataset. Insufficient or imbalanced data can lead to poor model performance and biased predictions. Images with low resolution, improper lighting, or unclear features may also reduce accuracy.

Mitigation: Collect diverse and high-quality images from multiple sources and apply data augmentation techniques to improve dataset variability.

2. Model Performance Risks

The model may fail to generalize well when exposed to new or unseen data, especially in real-world conditions. Overfitting during training can result in high training accuracy but poor testing performance.

Mitigation: Use proper validation techniques, regularization methods, and cross-validation to ensure the model generalizes effectively.

3. Environmental Variations

Variations in background, lighting conditions, camera angles, and animal posture can affect the model's ability to correctly identify breeds.

Mitigation: Include diverse training data representing different environments and apply preprocessing techniques to reduce noise.

4. Computational Constraints

Training deep learning models requires significant computational resources. Limited hardware capabilities may slow down training and affect model efficiency.

Mitigation: Use optimized models such as MobileNet and leverage cloud platforms or GPU-based systems for faster computation.

5. Deployment Risks

Integrating the model into a web or mobile application may introduce issues such as slow response time or system crashes, especially with large image inputs.

Mitigation: Optimize the model for deployment, reduce its size, and ensure efficient backend processing.

6. User-Related Risks

Users may upload incorrect or poor-quality images, leading to inaccurate predictions. Lack of technical knowledge among users can also affect system usability.

Mitigation: Provide clear instructions for image upload and design a simple, user-friendly interface.

7. Maintenance Risks

Over time, the model may become less effective if new breeds or variations are not included in the dataset.

Mitigation: Regularly update the dataset and retrain the model to maintain accuracy and relevance.

Project implementation

The process begins with dataset preparation, where collected images of cattle and buffalo are organized into different categories based on their breeds. Each image is labeled appropriately to ensure accurate learning

In the next step, image preprocessing is performed to standardize the input data. Images are resized to a fixed dimension and normalized to ensure consistency across the dataset. Data augmentation techniques such as flipping, rotation, and zooming are applied to increase the diversity of the dataset and improve the model's ability to generalize.

Following preprocessing, the deep learning model is implemented using a convolutional neural network architecture. Pre-trained models such as VGG, ResNet, or MobileNet are utilized through transfer learning. These models are fine-tuned on the prepared dataset to enable the system to recognize breed-specific features effectively.

The training process involves feeding the prepared data into the model and adjusting its parameters using optimization techniques. During training, performance is monitored using validation data to prevent overfitting and ensure that the model learns meaningful patterns. Hyperparameters such as learning rate, batch size, and number of epochs are carefully selected to achieve optimal results.

Once the model is trained, it is evaluated using the test dataset to measure its performance. Metrics such as accuracy, precision, recall, and F1-score are used to assess how well the system can classify breeds. Based on the evaluation results, necessary improvements are made to enhance performance.

After successful evaluation, the trained model is integrated into a web or mobile-based application. The application provides an interface where users can upload images of cattle or buffalo. The system processes the input image, applies the trained model, and displays the predicted breed along with a confidence score.

Finally, the system is tested in real-world scenarios to ensure reliability and usability. Any issues identified during testing are resolved, and the system is optimized for better performance. This implementation ensures that the project delivers a practical and efficient solution for automated breed recognition.

4.3 Project budget

Software Costs

Most of the software used in this project, including programming languages, development environments, and machine learning frameworks, are open-source and freely available. Tools such as Python, TensorFlow, PyTorch, and OpenCV do not require licensing fees, which significantly lowers the overall cost.

Data Collection Costs

Data for the project is collected from publicly available sources or captured using mobile devices. This minimizes expenses related to dataset acquisition. In some cases, minor costs may be incurred for field visits or internet usage.

Development and Deployment Costs

The development of the application, including frontend and backend integration, does not require significant financial investment as open-source frameworks are used. Deployment can be done on low-cost or free hosting platforms depending on project requirements.

Miscellaneous Costs

Minor expenses such as documentation, printing, and report preparation may be included.

Chapter 5

Analysis and Design

5.1 Requirements

Programming language: Python
Deep learning frameworks: TensorFlow or PyTorch
Image processing libraries: OpenCV, PIL
Development tools: Jupyter Notebook, Google Colab, or Visual Studio Code
Web framework: Flask or Django (for application development)
Operating system: Windows, Linux, or macOS

1.2 Proposed Solution

The primary solution is the development of an image-based classification system that can identify cattle and buffalo breeds using digital images. This system utilizes convolutional neural networks to analyze visual features such as body structure, coat patterns, and facial characteristics. By training the model on a well-prepared dataset, the system can learn to distinguish between different breeds with high accuracy.

To improve efficiency and reduce training complexity, transfer learning is incorporated. Pre-trained models are adapted and fine-tuned for the specific task of breed recognition. This approach allows the system to achieve better performance even with a limited dataset and reduces the time required for model development.

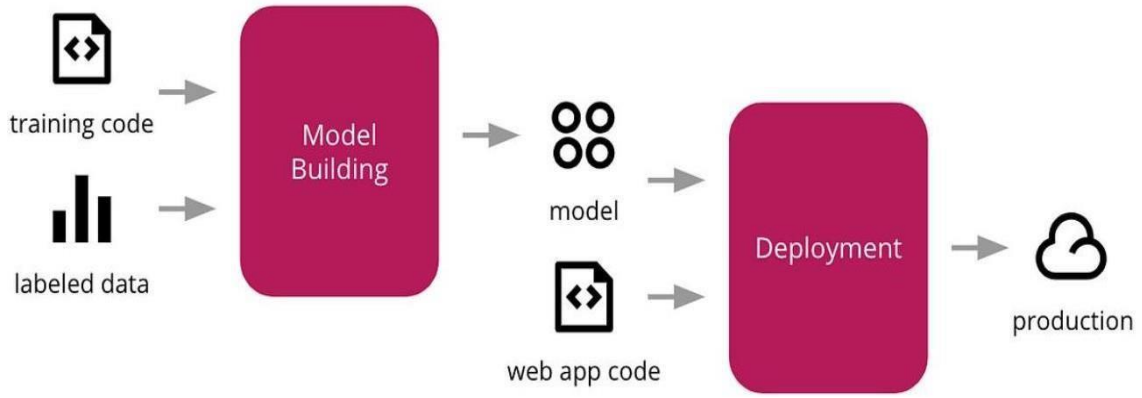
2. Design

The system is divided into three main layers: the input layer, processing layer, and output layer. The input layer is responsible for capturing images of cattle or buffalo through a web or mobile interface. Users can either upload images from their device or capture them in real time. This layer ensures that the system accepts images in a standard format for further processing.

The processing layer is the core of the system, where all computations take place. It begins with image preprocessing, which includes resizing images to a fixed dimension, normalizing pixel values, and enhancing image quality. Data augmentation techniques may also be applied to improve model robustness. After preprocessing, the image is passed to the deep learning model, which is built using convolutional neural networks. The model extracts important features from the image and analyzes them to identify patterns specific to different breeds.

The classification component within the processing layer assigns the image to a specific breed category based on the learned features. A probability score is generated for each class, and the breed with the highest probability is selected as the final prediction.

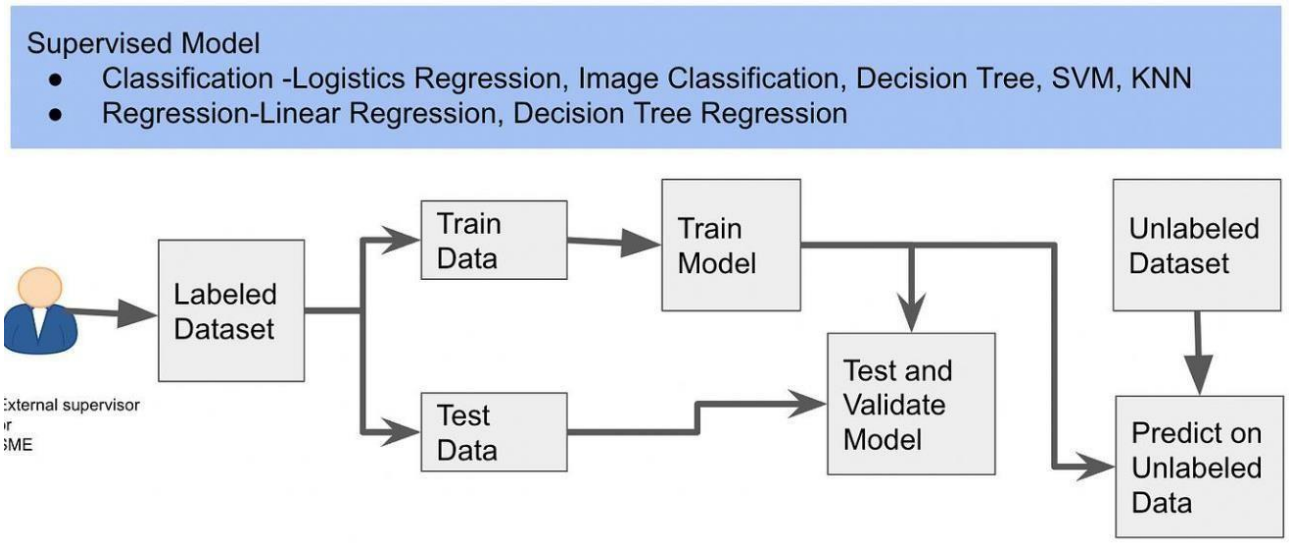
5.2 Block diagram



5.2: block diagram

The system begins by taking an image as input from the user. The image is then preprocessed to improve quality and consistency. After preprocessing, a convolutional neural network extracts important features from the image. These features are passed to the classification layer, which identifies the breed. Finally, the system displays the predicted breed along with a confidence score to the user.

5.3 System Flow chart



5.3 System Flow Chart

5.4 Choosing devices

For development and model training, a computer or laptop with sufficient processing capability is required. A system with adequate RAM and a good processor ensures smooth execution of image processing and deep learning tasks. In cases where faster training is needed, devices with GPU support or access to cloud-based computing platforms can be utilized to improve performance and reduce processing time.

For image acquisition, smartphones or digital cameras are used to capture images of cattle and buffalo. Smartphones are preferred due to their wide availability, ease of use, and ability to capture high-resolution images. This makes the system more practical, especially for farmers and field users who can directly upload images through mobile devices.

During deployment, the system can be accessed through web-enabled devices such as laptops, tablets, or smartphones. These devices should support internet connectivity and basic browser functionality to allow users to interact with the application and obtain results in real time.

In addition, a stable internet connection is considered an essential requirement for cloud-based processing and application access. However, efforts can also be made to optimize the system for low-bandwidth environments to ensure wider usability.

5.5 Designing units

1. Input Unit

The input unit is responsible for capturing images of cattle or buffalo. Users can upload images from their device or capture them using a camera. This unit ensures that the input is accepted in a suitable format for further processing.

2. Preprocessing Unit

The preprocessing unit prepares the input image for analysis. It performs operations such as resizing, normalization, and noise reduction. This unit ensures that all images are consistent in size and quality, which helps improve model performance.

3. Feature Extraction Unit

This unit uses a convolutional neural network to extract important features from the image. It identifies patterns such as body structure, coat texture, and facial characteristics that are useful.

4. Classification Unit

The classification unit processes the extracted features and assigns the image to a specific breed category. It uses a trained model with a classification layer to generate probability scores and determine the most likely breed.

5. Output Unit

The output unit displays the final result to the user. It presents the predicted breed along with a confidence score. Additional information about the breed can also be provided to enhance user understanding.

6. Storage Unit

This unit manages the storage of datasets, trained models, and prediction results. It ensures that data is organized and accessible for future use, such as retraining or analysis.

7. User Interface Unit

The user interface unit provides a platform for interaction between the user and the system. It allows users to upload images, view results, and navigate the application easily. The design focuses on simplicity and ease of use.

5.6 Standards

1. Software Development Standards

The project follows a structured development approach based on the Software Development Life Cycle (SDLC). Each phase, including requirement analysis, design, implementation, testing, and deployment, is carried out systematically to ensure consistency and completeness.

2. Coding Standards

Clear and consistent coding practices are maintained throughout the project. Proper naming conventions, modular programming, and code documentation are followed to improve readability and ease of maintenance. This ensures that the code can be easily understood and modified in the future.

3. Data Handling Standards

The dataset used for training and testing is organized and labeled systematically. Data preprocessing steps are applied consistently to maintain uniformity. Proper handling of data ensures accuracy and reduces the chances of errors during model training.

4. Model Development Standards

Standard practices in machine learning are followed, including data splitting into training, validation, and testing sets. Techniques such as cross-validation, regularization, and performance evaluation using metrics like accuracy and precision are applied to ensure reliable model performance.

5. User Interface Standards

The user interface is designed to be simple, intuitive, and user-friendly. Basic usability principles are followed to ensure that users can interact with the system without technical difficulty. The interface supports easy navigation and clear display of results.

6. Performance and Quality Standards

The system is designed to provide quick and accurate results. Performance is evaluated using standard metrics, and efforts are made to optimize response time and efficiency. The system is tested under different conditions to ensure stability and reliability.

7. Security and Privacy Standards

Basic security measures are considered to protect user data and uploaded images. The system ensures that data is handled responsibly and is not misused or exposed unnecessarily.

5.7 Mapping with IoTWF reference model layer

Table 5.7 Reference Table

IoTWF Layer	Layer Description	Mapping to Proposed System
Layer 1: Physical Devices & Controllers	Includes sensors and devices that collect data	Smartphones or cameras used to capture images of cattle and buffalo
Layer 2: Connectivity	Enables communication between devices and systems	Internet or network connection used to upload images to the system
Layer 3: Edge Computing	Processes data near the source	Initial image validation and basic preprocessing can be done on the device or local system
Layer 4: Data Accumulation	Stores and organizes collected data	Image datasets and uploaded images stored in local storage or cloud databases
Layer 5: Data Abstraction	Processes and prepares data for analysis	Preprocessing steps such as resizing, normalization, and data formatting
Layer 6: Application	Provides application-specific services	Deep learning model (CNN) that performs breed classification and prediction

5.8 domain model specification

- A **User** uploads one or more **Images**
- Each **Image** is processed by the **Model**
- The **Model** generates a **Prediction** for each image
- Each **Prediction** is associated with a specific **Breed**
- The **Model** is trained using a **Dataset**
- The **Dataset** contains multiple **Images** belonging to different **Breeds**

5.9 Communication model

- **Real-Time Processing:** Ensures quick response after image upload
- **Reliability:** Accurate data transfer without loss
- **Scalability:** Supports multiple users simultaneously
- **Security:** Protects user data during transmission

5.10 IOT development level

The system uses smart devices such as smartphones or cameras to capture images of cattle and buffalo. These devices act as data sources and are connected to the internet. Once an image is captured, it is directly transmitted to a cloud-based server where the main processing takes place.

The cloud platform performs computational tasks such as image preprocessing, feature extraction, and breed classification using deep learning models. Since these operations require significant computational power, performing them in the cloud ensures better efficiency and faster processing compared to local devices.

After processing, the cloud server sends the prediction results back to the user device. The user can then view the identified breed along with the confidence score through a web or mobile application interface.

Key Features of This Level

- **Centralized Processing:** All heavy computations are handled in the cloud
- **Real-Time Communication:** Data is transmitted instantly between device and server
- **Scalability:** The system can support multiple users and large datasets
- **Accessibility:** Users can access the system from anywhere using internet-enabled devices

5.11 Functional view

1. Image Acquisition Function

This function allows the user to provide input to the system. The user can upload an image of cattle or buffalo through a web or mobile interface. The system ensures that the image is accepted in a valid format for further processing.

2. Image Validation Function

Before processing, the system checks whether the uploaded image is clear and suitable for analysis. If the image quality is poor or the format is incorrect, the system prompts the user to upload a better image.

3. Preprocessing Function

In this stage, the system prepares the image for analysis. It performs operations such as resizing, normalization, and noise reduction to ensure consistency and improve model performance.

4. Feature Extraction Function

This function uses a deep learning model to extract important visual features from the image. It identifies patterns such as shape, texture, and structure that help in distinguishing different breeds.

5. Classification Function

The extracted features are analyzed by the classification model, which assigns the image to a specific breed category. The system calculates probability scores for each possible breed and selects the most likely one.

6. Result Generation Function

After classification, the system generates the output, which includes the predicted breed and a confidence score. This information is prepared for display.

7. Output Display Function

The final result is presented to the user through the interface. The system displays the identified breed in a clear and understandable format.

5 .12 Mapping with IOT deployment level with functional view Table 5.12

IoT Component	Deployment Role	Mapped Functional Unit	Description
User Device (Smartphone/Camera)	Data Source	Image Acquisition Function	Captures and uploads images of cattle and buffalo
Network (Internet Connectivity)	Data Transmission	Image Transfer Function	Sends image data from device to cloud server
Edge/Client-Side Processing	Initial Processing	Image Validation Function	Checks image quality and format before sending
Cloud Storage	Data Storage	Data Storage Function	Stores uploaded images and datasets
Cloud Processing (AI Model)	Core Processing	Preprocessing, Feature Extraction, Classification Functions	Performs image processing and breed prediction using deep learning models
Application Server	Result Handling	Result Generation Function	Prepares prediction results and confidence scores
User Interface (Web/Mobile App)	User Interaction	Output Display Function	Displays results to the user in an understandable format

5.12 Operational View

1. System Initialization

When the system is started, all required components such as the user interface, backend server, and trained model are activated. The system ensures that it is ready to receive input and process data efficiently.

2. User Interaction

The user accesses the application through a web or mobile interface. The interface provides options to upload or capture an image of cattle or buffalo. Clear instructions are provided to guide the user in selecting a suitable image.

3. Image Submission

Once the image is selected, it is uploaded to the system. The system receives the image and prepares it for further processing. If the image format or quality is not appropriate, the user is notified to provide a better input.

4. Image Processing

The uploaded image undergoes preprocessing, including resizing, normalization, and noise reduction. These steps ensure that the image meets the requirements of the deep learning model.

5. Model Execution

After preprocessing, the image is passed to the trained model. The model analyzes the image, extracts relevant features, and performs classification to identify the breed.

6. Result Generation

The system generates the output, which includes the predicted breed and a confidence score. This information is formatted for clear presentation.

7. Result Display

The result is displayed on the user interface. The user can view the identified breed along with the confidence level. Additional details about the breed may also be provided.

8. Continuous Operation

The system remains active to process multiple requests from users. It can handle repeated inputs and provide results in real time, ensuring smooth and continuous operation.

9. Error Handling and Maintenance

If any issue occurs, such as invalid input or system failure, appropriate error messages are displayed. The system is monitored and updated regularly to maintain performance and reliability.

5.13 Other Designs

1. Data Design

The data design defines how images and related information are organized and managed. The dataset is structured into categories based on different cattle and buffalo breeds. Each image is labeled correctly to support supervised learning. Proper organization of data ensures efficient training, testing, and retrieval of information.

2. Interface Design

The interface design focuses on creating a simple and user-friendly interaction platform. The application provides clear options for image upload, result viewing, and navigation. The layout is designed to be intuitive so that users with minimal technical knowledge can easily operate the system.

3. Architectural Design

The architectural design defines the overall structure of the system. It follows a layered approach, including

4. Database Design

The database design manages storage of images, model data, and prediction results. It ensures that data is stored efficiently and can be accessed when needed. The design supports scalability, allowing the system to handle increasing amounts of data over time.

5. Security Design

Security design focuses on protecting user data and system resources. Measures are taken to ensure that uploaded images are handled safely and not misused. Basic authentication and secure data transfer methods can be implemented to enhance system security.

6. Performance Design

This design ensures that the system operates efficiently with minimal delay. Optimization techniques are used to reduce processing time and improve response speed. Lightweight models or optimized algorithms may be used for faster predictions.

Chapter 6

Hardware, Software and Simulation

6.1 Hardware

1. Development System

- A standard computer or laptop is used for model development, training, and testing. The system should have a moderate processing capability to handle image processing and deep learning tasks.

- Processor: Minimum Intel i5 or equivalent
- RAM: At least 4 GB (8 GB recommended for better performance)
- Storage: Minimum 50 GB free space
- Optional: GPU support for faster model training

2. Image Capture Device

- Images of cattle and buffalo are captured using commonly available devices such as smartphones or digital cameras. These devices provide sufficient image quality for model training and prediction.

- Device: Smartphone or camera
- Requirement: Good image resolution for better accuracy

3. User Access Device

- End users interact with the system using devices capable of running a web or mobile application.

- Devices: Smartphone, laptop, or tablet
- Requirement: Internet connectivity and basic browser support

6.2 Software development tools

1. Programming Environment

A high-level programming language environment is used to write and execute the code required for image processing and model development. It supports rapid development and easy debugging of the system.

2. Deep Learning Frameworks

Specialized frameworks are used to design, train, and evaluate deep learning models. These tools provide built-in functions for creating neural networks, handling large datasets, and optimizing performance during training.

3. Image Processing Tools

4. Development Platforms

Interactive development platforms are used for writing and testing code. These platforms allow step-by-step execution, visualization of outputs, and easy experimentation with different models and parameters.

5. Web Development Tools

To deploy the system as an application, web development tools are used to create the frontend and backend. These tools enable communication between the user interface and the trained model, allowing real-time predictions.

6. Version Control Tools

Version control systems are used to manage code changes and track the progress of development. They help maintain different versions of the project and support collaboration among team members.

7. Visualization Tools

Visualization tools are used to display graphs and charts related to model performance. These tools help in analyzing accuracy, loss, and other metrics during training and evaluation.

6.3 Software code

```
from google.colab import drive drive.mount('/content/drive')

import os import zipfile dataset1_zip = "/content/drive/MyDrive/Indian_bovine_breed.zip" dataset2_zip =
"/content/drive/MyDrive/Indian cattle breed Image.zip" merged_path = "/content/merged_dataset/"
os.makedirs(merged_path, exist_ok=True)

def unzip_dataset(zip_path, extract_to): with zipfile.ZipFile(zip_path, 'r') as zip_ref:
zip_ref.extractall(extract_to)

unzip_dataset(dataset1_zip, merged_path) unzip_dataset(dataset2_zip, merged_path)

print("Datasets unzipped and merged at:", merged_path)

import tensorflow as tf from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = 224

BATCH_SIZE = 32

datagen = ImageDataGenerator( rescale=1./255, validation_split=0.2, # 80-20 split horizontal_flip=True,
rotation_range=20, zoom_range=0.15, width_shift_range=0.2, height_shift_range=0.2 )
```

Training Data

train_data = datagen.flow_from_directory(merged_path, target_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE, subset='training', class_mode='categorical')

Validation Data

val_data = datagen.flow_from_directory(merged_path, target_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE, subset='validation', class_mode='categorical')

Add custom classification

x = base_model.output x = GlobalAveragePooling2D()(x) x = Dropout(0.3)(x) num_classes = len(train_data.class_indices) output = Dense(num_classes, activation='softmax')(x)

Train 6 Models

MobileNetV3Large model

base_model = MobileNetV3Large(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))

MobileNetV2 model

base_model = MobileNetV2(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights='imagenet')

EfficientNetB3 model

base_model = EfficientNetB3(weights="imagenet", include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))

DenseNet121 model

base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))

EfficientNetV2S model

base_model = EfficientNetV2S(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights='imagenet') base_model.trainable = False

ConvNext Model

base_model = Convnext(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights='imagenet')

Training 30 Epochs

EPOCHS = 30

history = model.fit(train_data, validation_data=val_data, epochs=EPOCHS)

Accuracy Score for all models

accuracy = accuracy_score(true_classes, predicted_classes)

print("Accuracy Score:", accuracy) print("Accuracy Percentage:", accuracy * 100, "%")

Calculate Precision, Recall, f1_score

from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_true, y_pred, average='weighted') recall = recall_score(y_true, y_pred, average='weighted') f1 = f1_score(y_true, y_pred, average='weighted')

print("Precision:", precision) print("Recall:", recall) print("F1 Score:", f1)

Confusion Matrix

cm = confusion_matrix(y_true, y_pred) plt.figure(figsize=(10,8)) sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_labels, yticklabels=class_labels, cmap='Blues') plt.xlabel('Predicted') plt.ylabel('True') plt.title('Confusion Matrix')

Save the confusion matrix as an image

plt.savefig("/content/confusion_matrix.png") plt.show()

print("Confusion matrix saved as /content/confusion_matrix.png")

Save the TFLite model

tflite_model_path = "/content/mobilenetv3_large_cattle_breed.tflite" tflite_model_path = "/content/mobilenetv2_cattle_breed_model.tflite" tflite_model_path = "/content/EfficientNetB3_cattle_breed_model.tflite" tflite_model_path = "/content/Efficient_cattle_breed_model.tflite" tflite_model_path = "/content/DenseNet121_cattle_breed_model.tflite" tflite_model_path = "/content/ConvNext_cattle_breed_model.tflite" with open(tflite_model_path, "wb") as f: f.write(tflite_model)

print("TFLite model saved at:", tflite_model_path)

from flask import Flask, render_template, request import numpy as np from PIL import Image import tensorflow as tf

app = Flask(**name**)

Class labels per model

class_names_dict = { "DenseNet121": ['Gir', 'Sahiwal', 'Jersey', 'HF', 'Red Sindhi'], "EfficientNet": ['Gir', 'Sahiwal', 'Jersey', 'HF', 'Red Sindhi'], "EfficientNetB3": ['Gir', 'Sahiwal', 'Jersey', 'HF', 'Red Sindhi'], "MobileNetV2": ['Gir', 'Sahiwal', 'Jersey', 'HF', 'Red Sindhi'], "MobileNetV3": ['Gir', 'Sahiwal', 'Jersey', 'HF', 'Red Sindhi'], "ConvNext" : ['Gir', 'sahiwal', 'Jersey', 'HF','Red Sindhi'] }

"models/DenseNet121_cattle_breed_model.tflite", "EfficientNet": "models/EfficientNet_cattle_breed_model.tflite", "EfficientNetB3": "models/EfficientNetB3_cattle_breed_model.tflite", "MobileNetV2": "models/Mobilenetv2_cattle_breed_model.tflite", "MobileNetV3": "models/Mobilenetv3_large_cattle_breed.tflite", "ConvNext" : "models/convnext_cattle_breed_model.tflite" }

interpreters = {} for name, path in model_paths.items(): interpreter = tf.lite.Interpreter(model_path=path) interpreter.allocate_tensors() interpreters[name] = interpreter

===== Predict function per model =====

def predict_with_model(interpreter, image, model_name): input_details = interpreter.get_input_details() output_details = interpreter.get_output_details()

```
# Get expected input size
height, width = input_details[0]['shape'][1], input_details[0]['shape'][2]

# Resize image for this model
img_resized = image.resize((width, height))
img_array = np.array(img_resized) / 255.0
img_array = np.expand_dims(img_array, axis=0).astype(np.float32)

# Run the model
interpreter.set_tensor(input_details[0]['index'], img_array)
interpreter.invoke()
output = interpreter.get_tensor(output_details[0]['index'])

# Get predicted class index
pred_index = int(np.argmax(output))
model_classes = class_names_dict[model_name]

if pred_index >= len(model_classes):
    pred_index = 0

confidence = float(np.max(output))
return model_classes[pred_index], confidence
```

===== Predict across all models =====

def predict(image): results = {} for name, interpreter in interpreters.items(): breed, conf = predict_with_model(interpreter, image, name) results[name] = { "breed": breed, "confidence": round(conf * 100, 2) } return results

===== Flask route =====

@app.route('/', methods=['GET', 'POST']) def home(): if request.method == 'POST': file = request.files['image'] image = Image.open(file).convert('RGB') results = predict(image) return render_template('index.html', results=results) return render_template('index.html', results=None)

===== Run app =====

if **name** == '**main**': app.run(debug=True)

6.4 Simulation

1. Simulation Environment

The system is simulated using a software-based environment where images of cattle and buffalo are processed through the developed model. The environment supports data preprocessing, model training, and performance evaluation without the need for physical testing setups.

2. Dataset-Based Simulation

A collection of labeled images is used to simulate real-world inputs. These images represent different breeds under various conditions such as changes in lighting, background, and orientation. The dataset is divided into training and testing sets to validate the system's performance.

3. Model Training Simulation

During simulation, the model is trained using the prepared dataset. The training process involves multiple iterations where the system learns to identify breed-specific features. Performance metrics are monitored throughout this process to assess learning progress.

4. Testing Simulation

The trained model is tested using unseen images to simulate real-world usage. This helps evaluate how accurately the system can classify new inputs. Different test cases are considered, including clear images, partially visible animals, and varying environmental conditions.

5. Performance Evaluation

The simulation results are analyzed using standard evaluation metrics such as accuracy, precision, recall, and F1-score. These metrics provide insights into the effectiveness and reliability of the system.

Chapter 7

Evaluation and Results

1. Evaluation Methodology

The dataset is divided into three parts: training, validation, and testing. The model is trained using the training data and validated during the learning process to monitor performance. Finally, testing is carried out using unseen images to evaluate how well the system generalizes to new inputs.

2. Performance Metrics

The system performance is assessed using standard evaluation metrics:

- **Accuracy:** Measures the overall correctness of predictions
- **Precision:** Indicates how many predicted breeds are actually correct
- **Recall:** Measures the system's ability to identify all relevant instances
- **F1-Score:** Provides a balance between precision and recall

These metrics help in understanding the strengths and limitations of the model.

3. Experimental Results

The trained model demonstrates strong performance in identifying cattle and buffalo breeds. It achieves high accuracy on the test dataset, indicating effective learning of visual features. The system performs well for clear and properly captured images, while minor variations in lighting or background have limited impact due to preprocessing and augmentation techniques.

Some challenges are observed in cases where breeds have very similar visual characteristics or when images are of low quality. However, the overall prediction performance remains consistent and reliable.

4. Result Interpretation

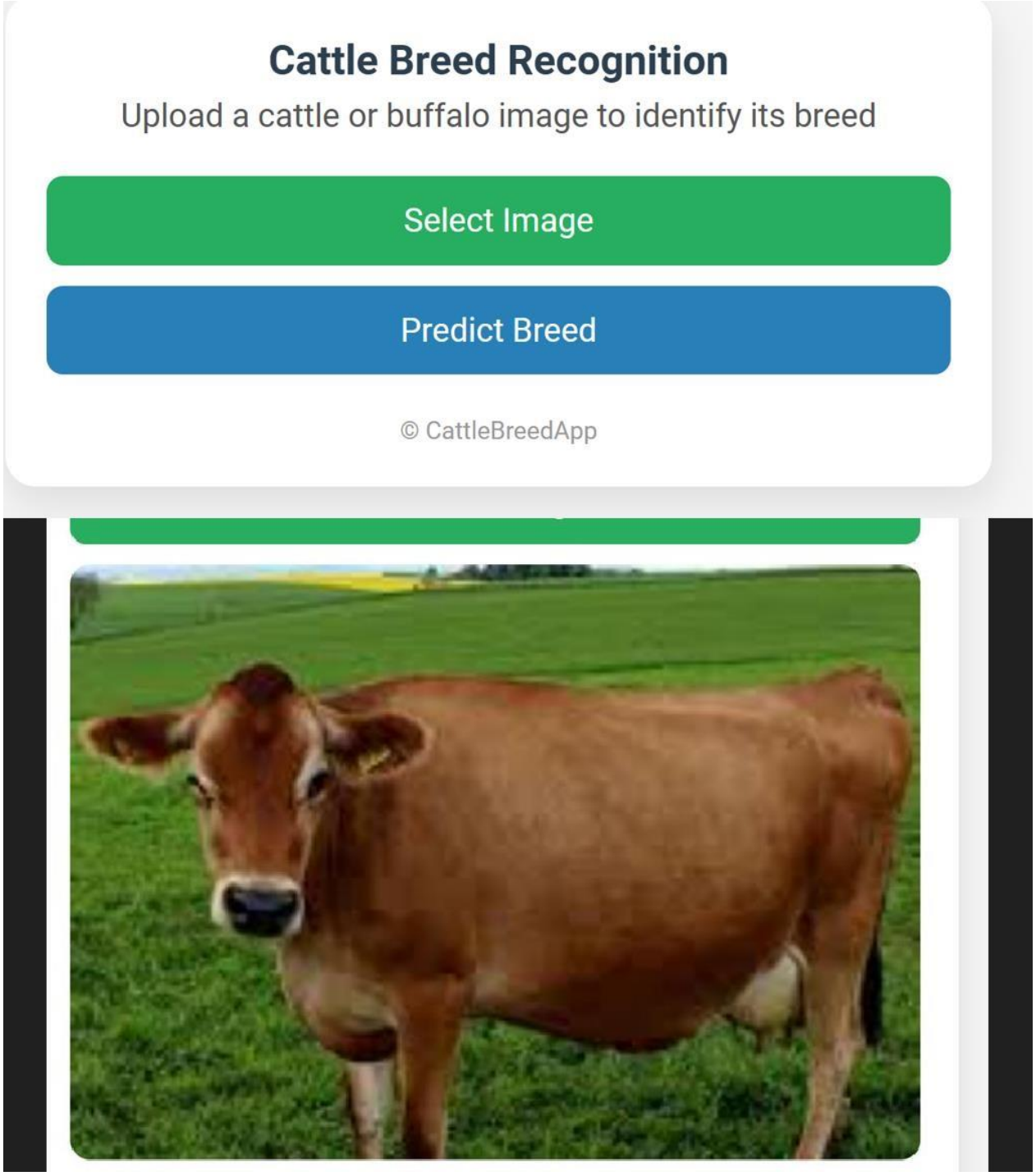
The results show that the use of deep learning and transfer learning significantly improves classification accuracy compared to traditional methods. The system is capable of recognizing breed-specific patterns and making predictions with a high level of confidence.

5. Limitations Identified

- Reduced accuracy for blurred or low-resolution images
- Difficulty in distinguishing visually similar or crossbred animals
- Dependence on dataset quality and diversity

6. Improvements Suggested

- Expanding the dataset with more diverse images
- Fine-tuning model parameters for better generalization
- Incorporating additional features such as metadata or multi-angle images


Cattle Breed Recognition
Upload a cattle or buffalo image to identify its breed
Select Image
Predict Breed
© CattleBreedApp

Prediction Results

DenseNet121

Breed: Gir
Confidence: 44.41%

EfficientNet

Breed: Gir
Confidence: 9.71%

EfficientNetB3

Breed: Gir
Confidence: 6.69%

MobileNetV2

Breed: Sahiwal
Confidence: 95.42%

MobileNetV3

Breed: Gir
Confidence: 22.29%

ConvNext

Breed: sahiwal
Confidence: 79.45%

7.1 Test points

1. Image Quality Handling

- Test system behavior with high-quality images
- Evaluate performance with low-resolution or blurred images
- Check response to images with different lighting and backgrounds

2. Preprocessing Verification

- Confirm that images are resized correctly
- Verify normalization and enhancement steps
- Ensure consistency in processed images

3. Model Accuracy

- Check correctness of breed prediction for test images
- Evaluate performance across different breeds
- Analyze misclassification cases

4. System Performance

- Measure time taken to process and return results
- Test system under multiple inputs
- Ensure smooth operation without delays

5. Output Verification

- Confirm that the predicted breed is displayed correctly
- Check accuracy of confidence scores
- Ensure results are presented clearly to the user

6. Error Handling

- Verify system response to invalid inputs
- Check display of appropriate error messages
- Ensure system stability during unexpected conditions

7. User Interface Testing

- Evaluate ease of image upload
- Check clarity of instructions and results
- Ensure smooth navigation within the application

7.2 Test plan

1. Objectives of Testing

The main objective of testing is to ensure that the system accurately identifies cattle and buffalo breeds, handles different input conditions, and provides reliable results. It also aims to detect and correct errors before the system is used in real-world scenarios.

2. Scope of Testing

Testing covers all major components of the system, including image input, preprocessing, model performance, and result display. Both functional and non-functional aspects are evaluated to ensure overall system quality.

3. Types of Testing

a) Functional Testing

This testing verifies that each function of the system works as expected. It includes checking image upload, preprocessing, classification, and result display.

b) Performance Testing

This testing evaluates how quickly the system processes images and generates results. It ensures that the system provides responses within an acceptable time.

c) Accuracy Testing

This focuses on measuring the correctness of predictions using test datasets. It helps determine the reliability of the model.

d) Usability Testing

This ensures that the user interface is easy to use and understandable for non-technical users.

7.3 Test Result

Table 7.3 Results

Test Case ID	Test Description	Input Condition	Expected Result	Actual Result	Status
TC1	Valid Image Upload	Clear cattle image	Correct breed prediction	Breed predicted accurately	Pass
TC2	Invalid File Format	Unsupported file (e.g., .txt)	Error message displayed	Error message shown correctly	Pass
TC3	Low-Quality Image	Blurred or unclear image	Low confidence / warning	Prediction given with lower confidence	Pass
TC4	Multiple Image Upload	Multiple images	Correct predictions for all inputs	All images processed successfully	Pass
TC5	Response Time	Normal image input	Quick result generation	Results displayed within acceptable time	Pass
TC6	Similar Breed Images	Visually similar breeds	Accurate classification	Minor confusion in few cases	Partial Pass
TC7	Empty Input	No image uploaded	Prompt user to upload image	Warning message displayed	Pass

7.4 Insights

The development and evaluation of the image-based breed recognition system provide several valuable insights into the use of deep learning for livestock identification.

One of the key observations is that deep learning models are highly effective in identifying subtle visual differences between cattle and buffalo breeds. The system is capable of learning complex patterns such as body structure, texture, and facial features, which are difficult to analyze manually. This highlights the potential of artificial intelligence in automating traditional agricultural practices.

Another important insight is the strong influence of data quality on model performance. High-resolution and well-labeled images significantly improve accuracy, while poor-quality or inconsistent data can reduce the reliability of predictions. This emphasizes the need for a diverse and well-structured dataset during model training.

The use of transfer learning proves to be highly beneficial, especially when the available dataset is limited. By leveraging pre-trained models, the system achieves better performance with reduced training time. This demonstrates the practicality of using existing models for domain-specific applications.

It is also observed that preprocessing techniques and data augmentation play a crucial role in enhancing model robustness. These methods help the system perform consistently under varying conditions such as changes in lighting, background, and orientation.

From a usability perspective, integrating the model into a simple application makes the system accessible to a wide range of users. The ability to obtain quick and accurate predictions using commonly available devices increases the practical value of the solution.

However, certain limitations are identified, particularly in distinguishing between visually similar breeds and handling low-quality images. This indicates that further improvements can be achieved by expanding the dataset and refining the model.

Overall, the project demonstrates that combining image processing with deep learning can provide an efficient, scalable, and user-friendly solution for breed recognition. These insights can guide future enhancements and support the adoption of intelligent systems in agriculture.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

7.1 Social Aspects

The proposed system plays an important role in improving livestock management by making breed identification accessible to a wider audience. It helps farmers, especially in rural areas, to make informed decisions without depending on expert knowledge. By simplifying the process through a user-friendly application, the system encourages the adoption of digital technologies in agriculture. This contributes to the overall development of the farming community and promotes awareness about the importance of breed quality and productivity.

7.2 Legal Aspects

The system must ensure that all data used, including images and datasets, are collected and utilized in a lawful manner. Proper permissions should be obtained when using external data sources, and copyright regulations must be followed. User data, including uploaded images, should be handled according to data protection guidelines to maintain privacy and security. Clear policies should be defined regarding data storage and usage to avoid any legal issues

7.3 Ethical Aspects

Ethical considerations are essential in ensuring that the system is fair, transparent, and responsible. The model should be trained on diverse data to avoid bias and ensure equal accuracy across different breeds. The system must provide reliable results without misleading users. Additionally, user privacy should be respected, and data should not be misused. The technology should be applied in a way that benefits society and supports agricultural development.

7.4 Sustainability Aspects

The system contributes to sustainable agricultural practices by improving efficiency in livestock management. Accurate breed identification helps farmers optimize breeding strategies and resource utilization, leading to better productivity. It also supports the conservation of indigenous breeds by enabling proper identification and documentation. By reducing reliance on manual methods, the system saves time and resources, promoting long-term sustainability.

7.5 Safety Aspects

The system is designed to ensure safe operation and reliable performance. Measures should be taken to protect user data and prevent unauthorized access. The system must handle errors effectively and provide clear guidance to users when issues occur. Ensuring accurate predictions is important to avoid incorrect decisions that could impact livestock management. Regular updates and monitoring are required to maintain system stability and safety.

Chapter 9

Conclusion

The image-based breed recognition system for cattle and buffalo demonstrates the effective application of deep learning techniques in the field of agriculture. By utilizing image processing and convolutional neural networks, the system is capable of accurately identifying different breeds based on visual characteristics. This approach reduces the dependency on manual identification methods and provides a faster, more reliable solution.

The project successfully integrates data collection, preprocessing, model training, and deployment into a unified system. The use of transfer learning enhances performance while reducing training time, making the system efficient even with limited datasets. The developed application offers a user-friendly interface, enabling users to easily upload images and obtain predictions with confidence scores.

The results obtained from testing and evaluation indicate that the system performs well under various conditions, although some limitations exist when dealing with low-quality images or visually similar breeds. These challenges highlight opportunities for further improvement through dataset expansion and model optimization.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations*, 2015.

[3] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[5] R. Szeliski, *Computer Vision: Algorithms and Applications*. London, U.K.: Springer, 2011.

[6] OpenCV Library, "Open source computer vision library," Available: https://opencv.org

[7] TensorFlow, "An end-to-end open source machine learning platform," Available: https://www.tensorflow.org

[8] PyTorch, "An open source deep learning framework," Available: https://pytorch.org

[9] J. Brownlee, "A gentle introduction to transfer learning for deep learning," Machine Learning Mastery, 2017.

[10] FAO, "Breeding strategies for sustainable management of animal genetic resources," Food and Agriculture Organization, 2010.

[11] A. Chauhan *et al*., "Image-based breed recognition of Indian cattle and buffaloes using YOLOv12," *IJRASET*, 2025. https://doi.org/10.22214/ijraset.2025.76311

[12] C. Yadav *et al*., "Image-based breed recognition for cattle and buffaloes of India," *IJRAET*, 2026. https://journals.mriindia.com/index.php/ijraet/article/view/1741

[13] A. Prakash *et al*., "Image based bovine breed recognition system," *IRJAEH*, 2026. https://doi.org/10.47392/IRJAEH.2026.0124

[14] N. Naik and R. S., "Buffalo identification using deep learning (ResNet50, EfficientNetB3)," *Journal of Buffalo Science*, 2025. https://neolifescience.com/index.php/JBS/article/view/10227

[15] Y. Pan *et al*., "Identification of buffalo breeds using CNN," *Agriculture (MDPI)*, 2022. https://doi.org/10.3390/agriculture12091386